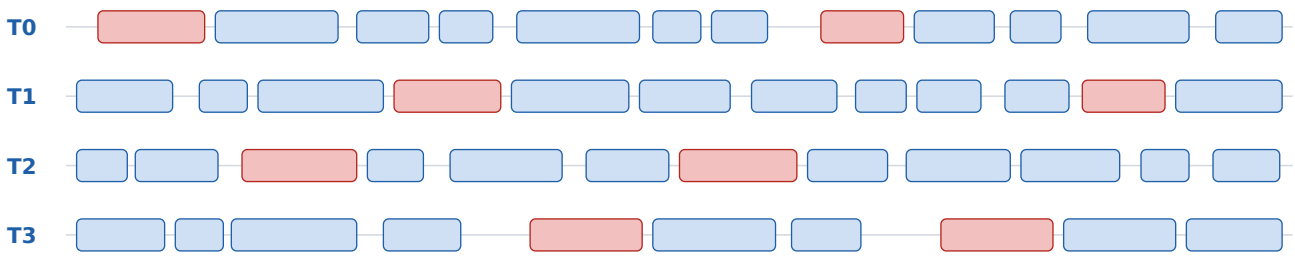
WEEK 4

Concurrency in Modern C++

Threads, locks, condition variables, futures, atomics and the memory model — from first principles to a working thread pool



Four threads over time. Red segments are critical sections — the moments where threads must take turns.

Syllabus coverage: `std::thread` · mutexes · `lock_guard` · `unique_lock` · condition variables · futures/promises · atomics · race conditions · deadlocks · critical sections · producer-consumer · thread-safe APIs. **Project:** a thread pool with worker threads, a task queue, synchronization and clean shutdown.

Every complete program in this guide was compiled with `g++ -std=c++20 -Wall -Wextra` (GCC 13) and run under ThreadSanitizer with no reports, except `04_race.cpp`, which is broken on purpose.

Contents

0. How to Use This Guide . . . 5

Learning objectives . . . 5

Suggested 7-day schedule . . . 5

How to read the code . . . 6

1. The Mental Model . . . 7

1.1 Why concurrency at all? . . . 7

1.2 Processes vs threads . . . 7

1.3 Who decides what runs? . . . 7

1.4 How much faster can it get? Amdahl’s law . . . 8

2. Creating and Managing Threads: std::thread . . . 9

2.1 Starting a thread . . . 9

2.2 join, detach, and the terminate rule . . . 10

2.3 How arguments are passed . . . 10

2.4 Exceptions inside a thread . . . 11

2.5 std::jthread and cooperative cancellation (C++20) . . . 11

2.6 Useful odds and ends . . . 12

Exercises §2 . . . 12

3. Race Conditions, Data Races and Critical Sections . . . 13

3.1 Why ++counter is not one step . . . 13

3.2 Two different things: data race vs race condition . . . 14

3.3 Critical sections and invariants . . . 15

Exercises §3 . . . 15

4. Mutexes and Locks . . . 16

4.1 std::mutex . . . 16

4.2 The RAII lock types . . . 17

4.3 Other mutex types . . . 18

4.4 Using mutexes well . . . 18

Exercises §4 . . . 19

5. Deadlocks (and Their Cousins) . . . 21

5.1 What a deadlock is . . . 21

5.2 The four necessary conditions (Coffman, 1971) . . . 21

5.3 The canonical example and its fix . . . 22

5.4 Prevention techniques . . . 22

5.5 Livelock and starvation . . . 23
5.6 Debugging a hang . . . 23
Exercises §5 . . . 23

6. Condition Variables . . . 24

6.1 The problem: waiting for something to become true . . . 24
6.2 The canonical pattern . . . 24
6.3 The rules — and why each one exists . . . 25
6.4 notify_one vs notify_all; notify inside or outside the lock . . . 25
6.5 Timed waits . . . 25
Exercises §6 . . . 26

7. The Producer–Consumer Pattern . . . 27

7.1 Idea . . . 27
7.2 Why bounded? . . . 27
7.3 A complete, shutdown-aware implementation . . . 28
7.4 Design decisions worth understanding . . . 29
Exercises §7 . . . 29

8. Futures, Promises, packaged_task and async . . . 30

8.1 The idea: a one-shot channel for a result . . . 30
8.2 The four tools, compared . . . 31
8.3 Important details . . . 31
8.4 std::async pitfalls . . . 31
Exercises §8 . . . 32

9. Atomics and the Memory Model . . . 33

9.1 What std::atomic gives you . . . 33
9.2 Compare-and-swap: the building block of lock-free code . . . 33
9.3 Why ordering matters . . . 34
9.4 The three orderings you need . . . 34
9.5 Spinlocks vs mutexes . . . 36
9.6 False sharing: a performance bug with no wrong answer . . . 36

10. Designing Thread-Safe APIs . . . 37

10.1 Thread-safe implementation ≠ thread-safe interface . . . 37
10.2 Guidelines . . . 38
10.3 Safe one-time initialization and per-thread state . . . 38
10.4 Other C++20 coordination tools (beyond the syllabus, good to know) . . . 39

11. Testing and Debugging Concurrent Code 41

11.1 ThreadSanitizer (your most important tool this week) 41

11.2 Writing tests that can actually fail 41

11.3 Debugging 41

Exercises §9–§11 41

12. Project: A Thread Pool 43

12.1 What a thread pool is and why it exists 43

12.2 Requirements 43

12.3 The interface (compiles as-is) 44

12.4 Design walkthrough (read, then implement) 44

12.5 Milestones 45

12.6 Test plan 46

12.7 Benchmark and analysis 46

12.8 Traps (check your code against each) 47

12.9 Stretch goals 47

12.10 Self-review rubric 47

13. Review: Cheat Sheet and Self-Check 49

13.1 “I need to …” → use 49

13.2 Golden rules 49

13.3 Self-check questions 50

13.4 Answers 50

14. Curated Resources 51

14.1 Core (use these this week) 51

14.2 Atomics and the memory model 51

14.3 Design, practice and going further 52

14.4 Looking ahead 52

0. How to Use This Guide

Week 4 is where C++ stops being a language you read top to bottom and becomes a system of independent actors that share memory. That shift is mental before it is syntactic. Most bugs in concurrent code are not typos; they are wrong assumptions about *what can happen between two lines*. So this guide spends real time on the mental model, and only then on APIs.

Learning objectives

By the end of the week you should be able to:

- Start, join and manage threads safely, including passing arguments and handling exceptions.
- Explain precisely what a **data race** is, why it is undefined behaviour, and how it differs from a **race condition**.
- Protect shared state with mutexes using RAII locks (`lock_guard`, `unique_lock`, `scoped_lock`, `shared_lock`).
- Recognise, prevent and debug **deadlocks**.
- Block efficiently with **condition variables**, and handle spurious and lost wake-ups correctly.
- Build a **bounded producer-consumer queue** with clean shutdown.
- Return results and exceptions across threads with **futures, promises,** `packaged_task` **and** `async`.
- Use **atomics** correctly, and explain sequential consistency vs acquire/release vs relaxed ordering.
- Design **thread-safe APIs** whose operations cannot be misused, and find bugs with ThreadSanitizer.
- Implement a **thread pool** from an empty file.

Suggested 7-day schedule

Day	Read in this guide	Watch / read externally	Do
1	§1 Mental model, §2 std::thread	OSTEP ch. 26–27; Williams ch. 1–2	Exercises 2.x; run every example
2	§3 Races, §4 Mutexes	Williams ch. 3; O'Dwyer B2B Concurrency (first half)	Break and fix the racy counter; TSan it
3	§5 Deadlock, §6 Condition variables	OSTEP ch. 30, 32	Bank-transfer + handshake exercises
4	§7 Producer-consumer, §8 Futures	Williams ch. 4	Write BoundedQueue from memory
5	§9 Atomics & memory model, §10 Thread-safe APIs	Preshing (acquire/release posts); Sutter "atomic<> Weapons" part 1	Atomic exercises; false-sharing benchmark
6	§11 Tools, §12 Project spec	Williams ch. 9.1 (thread pools)	Thread pool milestones 1–3
7	§12 Project, §13 Review	—	Milestones 4–6, tests, self-review rubric

How to read the code

Full programs are shown with a dark title bar naming the file. Short fragments have no title bar. Compile every full program yourself; on Linux/macOS use:

```
g++ -std=c++20 -Wall -Wextra -g -O1 -pthread file.cpp -o prog            # normal
g++ -std=c++20 -Wall -Wextra -g -O1 -pthread -fsanitize=thread file.cpp  # race detector
```

TRY IT

Run every example **several times** and on a machine with several cores. Concurrency bugs are probabilistic: a broken program can print the right answer a thousand times in a row. One run proves nothing.

1. The Mental Model

1.1 Why concurrency at all?

There are two different reasons to use more than one thread, and they lead to different designs:

- **Concurrency for responsiveness / structure.** Keep doing useful work while something else waits: a UI that stays responsive during a download, a server that handles client B while client A's disk read is in flight. This helps even on a single core.
- **Parallelism for throughput.** Split a CPU-heavy job across cores so it finishes sooner. This only helps if you actually have idle cores and the work can be divided.

Concurrency is about **structure**: several tasks are in progress at overlapping times. *Parallelism* is about **execution**: several tasks literally run at the same instant on different cores. A single-core machine can be concurrent (by time-slicing) but not parallel.

1.2 Processes vs threads

A **process** is a running program with its own virtual address space. Two processes cannot see each other's variables. A **thread** is an independent flow of execution *inside* a process. All threads of a process share the same address space: the same globals, the same heap, the same code. What each thread owns privately is its **stack** (local variables, return addresses) and its **registers** (including the program counter).

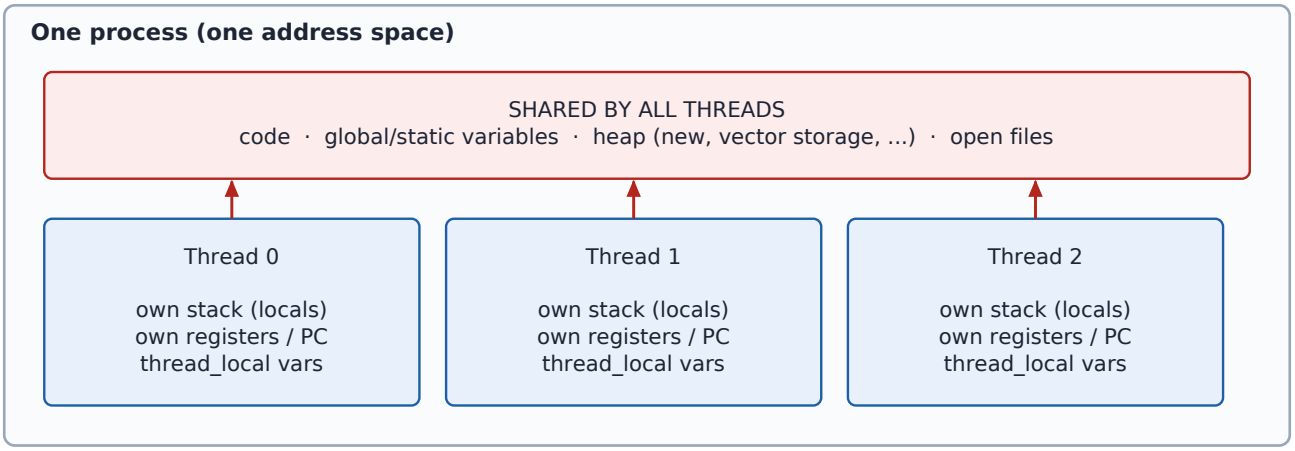


Figure 1. Threads share everything except their stacks and registers. Shared = needs coordination.

KEY IDEA

Anything reachable from more than one thread — a global, a heap object, a local passed by reference — is **shared state**. Shared state that is **written** by at least one thread is where every concurrency problem in this guide comes from.

1.3 Who decides what runs?

The operating system **scheduler** decides which thread runs on which core and for how long. It can **preempt** a thread at almost any instruction boundary: between loading a variable and storing it back, in the middle of your `if`, anywhere. You have no control over this order and must assume the worst possible interleaving will eventually happen.

On top of that, two more agents reorder your code: the **compiler** (which may reorder, merge or delete memory accesses as long as a *single-threaded* program could not tell), and the **CPU** (which executes out of order and keeps values in per-core caches and store buffers). C++'s **memory model** (§9) is the contract that tells you which orderings you can rely on — and the answer is: only those you establish with synchronization.

SYSTEMS CONNECTION

This is exactly the OS course's scheduling chapter and the architecture course's cache-coherence chapter seen from the programmer's side. Context switches, preemption, MESI coherence and store buffers all show up later as *why* the rules in this guide exist.

1.4 How much faster can it get? Amdahl's law

If a fraction p of a program's work can run in parallel and the rest is serial, then with N threads the best possible speedup is:

$$speedup(N) = 1 / ((1 - p) + p / N)$$

With $p = 0.9$, 8 threads give at most 4.7×, and infinitely many give at most 10×. The serial part dominates. In practice you get less than Amdahl predicts, because threads also pay for synchronization, contention on locks, cache-line ping-pong (§9.6) and memory bandwidth. Keep this in mind: it is precisely the question your Week 6 capstone asks ("why does scaling stop?").

2. Creating and Managing Threads: std::thread

2.1 Starting a thread

`std::thread` (header `<thread>`) starts running the callable you give it **immediately** on a new OS thread. Anything callable works: a function, a lambda, a function object, or a member function plus an object.

01_hello_threads.cpp

```cpp
// 01_hello_threads.cpp  --  four ways to give a thread work
#include <iostream>
#include <string>
#include <thread>
#include <vector>
void greet(const std::string& who, int times) {
    for (int i = 0; i < times; ++i)
        std::cout << "hello, " + who + "\n";   // one write per line
}
struct Counter {
    int value = 0;
    void add(int n) { value += n; }
};
struct Worker {                        // a callable object (functor)
    int id;
    void operator()() const {
        std::cout << "functor worker " + std::to_string(id) + "\n";
    }
};
int main() {
    // 1. Free function + arguments (arguments are COPIED into the thread)
    std::thread t1(greet, "alice", 2);
    // 2. Lambda
    std::thread t2([] { std::cout << "hello from a lambda\n"; });
    // 3. Function object
    std::thread t3(Worker{7});
    // 4. Member function: pass pointer-to-member + object pointer
    Counter c;
    std::thread t4(&Counter::add, &c, 42);
    t1.join(); t2.join(); t3.join(); t4.join();   // wait for all four
    std::cout << "counter = " << c.value << "\n"; // safe: t4 has finished
    // Launch N threads and keep them in a vector (std::thread is move-only)
    unsigned n = std::thread::hardware_concurrency();   // may return 0!
    if (n == 0) n = 4;
    std::vector<std::thread> pool;
    for (unsigned i = 0; i < n; ++i)
        pool.emplace_back([i] {
            std::cout << "thread " + std::to_string(i) + "\n";
        });
    for (auto& t : pool) t.join();
}
```

Things to notice:

- Output lines from different threads interleave unpredictably. We build each line as one string and write it with one `<<` so lines don't get torn apart mid-line. `std::cout` itself is safe to use from several threads (no data race on the stream), but separate `<<` calls from different threads can interleave.
- `hardware_concurrency()` is a *hint* of how many hardware threads exist and is allowed to return 0.

- `std::thread` is **move-only**: you can put it in a vector with `emplace_back` or `std::move`, but not copy it. Each `std::thread` object owns exactly one running thread.

2.2 join, detach, and the terminate rule

Every `std::thread` that represents a running thread is **joinable**. Before the `std::thread` object is destroyed you must decide its fate:

- `join()` — block until the thread finishes. After it returns, everything the thread wrote is visible to you (join is a synchronization point).
- `detach()` — let it run on independently; the `std::thread` object no longer refers to it. You can never join it or find out when it ends.

PITFALL

If a joinable `std::thread` is destroyed (goes out of scope, or is assigned over) the program calls `std::terminate()` and dies. This is deliberate: the standard refuses to guess whether you wanted to wait or abandon it. This is easy to hit when an exception is thrown between creating the thread and calling `join()`.

```
void risky() {
    std::thread t(do_work);
    something_that_may_throw();   // if this throws, t is destroyed while
    t.join();                     // joinable -> std::terminate()
}
```

The fix is the same as for any resource in Week 2: **RAII**. Wrap the thread in an object whose destructor joins. C++20 ships exactly that as `std::jthread`. In C++17 and earlier, write a tiny guard:

```
class JoinGuard {                         // C++17-style RAII joiner
public:
    explicit JoinGuard(std::thread t) : t_(std::move(t)) {}
    ~JoinGuard() { if (t_.joinable()) t_.join(); }
    JoinGuard(const JoinGuard&) = delete;
    JoinGuard& operator=(const JoinGuard&) = delete;
private:
    std::thread t_;
};
```

PITFALL

Detached threads are rarely what you want. A detached thread that still touches a local variable of the function that launched it reads freed stack memory once that function returns. And when `main` returns, the process ends and detached threads are killed mid-work with no cleanup. Prefer join.

2.3 How arguments are passed

The `std::thread` constructor **copies (or moves) every argument** into storage owned by the new thread, and then calls your function with those copies. This protects you from dangling references by default — but it surprises people whose function takes a reference.

02_args.cpp

```cpp
// 02_args.cpp  --  how arguments reach a thread
#include <functional>   // std::ref
#include <iostream>
#include <memory>
#include <thread>
void increment(int& x) { ++x; }
void consume(std::unique_ptr<int> p) { std::cout << *p << "\n"; }
int main() {
    int value = 0;
    // std::thread t(increment, value);   // ERROR: won't compile.
    // The thread stores a COPY of value; a copy cannot bind to int&.
    std::thread t(increment, std::ref(value));   // pass a real reference
    t.join();
    std::cout << "value = " << value << "\n";     // prints 1
    // Move-only arguments must be moved in:
    auto p = std::make_unique<int>(99);
    std::thread t2(consume, std::move(p));        // p is now empty
    t2.join();
    // Capturing by reference in a lambda works too -- but YOU must
    // guarantee the referenced object outlives the thread.
    std::thread t3([&value] { value += 10; });
    t3.join();                                    // value still alive: OK
    std::cout << "value = " << value << "\n";     // prints 11
}
```

PITFALL

A classic dangling bug: `char buf[64]; ... std::thread t(f, buf); t.detach();` — the thread receives a `char*` to `buf`, and the conversion to `std::string` may happen *on the new thread*, after `buf` is gone. Convert first: `std::thread t(f, std::string(buf));`.

2.4 Exceptions inside a thread

If an exception escapes the top-level function of a thread, `std::terminate()` is called. It does **not** propagate to the thread that called `join()`. You have two options: catch everything inside the thread and report it somehow, or — much better — run the work through `std::async`/`std::packaged_task`/`std::promise` so the exception is stored in a **future** and rethrown by `get()` (§8). Your thread pool will rely on this.

2.5 std::jthread and cooperative cancellation (C++20)

`std::jthread` is a `std::thread` that (1) joins automatically in its destructor and (2) carries a **stop token**. Calling `request_stop()` sets a flag that the thread can poll. Nothing is killed forcibly; the thread must check the token and exit by itself. That is called **cooperative cancellation**, and it is the only safe kind: forcibly killing a thread could leave a mutex locked forever.

03_jthread.cpp

```cpp
// 03_jthread.cpp  --  C++20 std::jthread: auto-join + cooperative stop
#include <chrono>
#include <iostream>
#include <stop_token>
#include <thread>
using namespace std::chrono_literals;
int main() {
    std::jthread ticker([](std::stop_token st) {
        int n = 0;
        while (!st.stop_requested()) {        // poll the stop flag
            std::cout << "tick " << n++ << "\n";
```

```
            std::this_thread::sleep_for(100ms);
        }
        std::cout << "ticker: stop requested, cleaning up\n";
    });
    std::this_thread::sleep_for(350ms);
    // No explicit join needed: ~jthread() calls request_stop() then join().
    // We could also call ticker.request_stop() ourselves earlier.
}
```

If the jthread's callable accepts a `std::stop_token` as its first parameter, jthread passes it in automatically. A `std::condition_variable_any` can even wait on a stop token, so a sleeping thread wakes up when stop is requested.

2.6 Useful odds and ends

Facility	What it does
`std::this_thread::get_id()` / `t.get_id()`	Opaque, comparable, printable thread ID. Useful for logging.
`std::this_thread::sleep_for(d)` / `sleep_until(tp)`	Block the calling thread for at least a duration. Never use sleeps to "fix" races.
`std::this_thread::yield()`	Hint to the scheduler: let another thread run now.
`std::thread::hardware_concurrency()`	Number of hardware threads (hint; may be 0).
`t.native_handle()`	The underlying `pthread_t` (POSIX) or `HANDLE` (Windows), for OS-specific tuning such as CPU affinity.

SYSTEMS CONNECTION

On Linux, `std::thread` is a thin wrapper over `pthread_create`, and each thread is a kernel task visible in `top -H` or `ps -eLf`. Creating a thread costs microseconds and a stack (often 8 MB of virtual memory). That cost is the reason thread **pools** exist: create the threads once, reuse them for thousands of tasks.

Exercises §2

1. Launch 8 threads that each print their index and `get_id()`. Run it 5 times. Is the order stable?
2. Write `void fill(std::vector<int>& v, int value)` and run it on a thread. Get the compile error without `std::ref`, read it, then fix it.
3. Throw an exception from a thread's function without catching it. Observe the crash. Then rewrite using `std::async` and catch it in `main`.
4. Write the `JoinGuard` class and demonstrate that an exception between creation and join no longer terminates.
5. Use `std::jthread` to build a "heartbeat" thread that prints every 200 ms until `main` finishes its work.

3. Race Conditions, Data Races and Critical Sections

3.1 Why `++counter` is not one step

To the CPU, `++counter` on a shared variable is three steps: **load** the value from memory into a register, **add** one, **store** it back. If two threads do this at the same time, the scheduler can interleave the steps:

Step	Thread A	Thread B	counter in memory
1	load counter → regA = 5		5
2		load counter → regB = 5	5
3	regA = regA + 1 = 6		5
4		regB = regB + 1 = 6	5
5	store regA → counter		**6**
6		store regB → counter	**6** (one increment lost)

Two increments happened; the counter went up by one. This is a **lost update**, the most common symptom of unsynchronized sharing.

04_race.cpp

```cpp
// 04_race.cpp  --  a data race, and two fixes
#include <atomic>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
constexpr int kThreads = 4;
constexpr int kIters   = 1'000'000;
template <class F>
void run_parallel(F f) {
    std::vector<std::thread> ts;
    for (int i = 0; i < kThreads; ++i) ts.emplace_back(f);
    for (auto& t : ts) t.join();
}
int main() {
    // BROKEN: data race -> undefined behaviour. Usually prints < 4000000.
    long racy = 0;
    run_parallel([&] { for (int i = 0; i < kIters; ++i) ++racy; });
    std::cout << "racy   = " << racy << "\n";
    // FIX 1: mutex makes ++ a critical section
    long locked = 0;
    std::mutex m;
    run_parallel([&] {
        for (int i = 0; i < kIters; ++i) {
            std::lock_guard<std::mutex> lk(m);
            ++locked;
        }
    });
    std::cout << "locked = " << locked << "\n";
    // FIX 2: atomic read-modify-write, no lock needed
    std::atomic<long> atomic_count{0};
    run_parallel([&] {
        for (int i = 0; i < kIters; ++i) ++atomic_count;
    });
    std::cout << "atomic = " << atomic_count << "\n";
}
```

Two surprising facts appear the moment you run this:

- **On a single core, or with optimisations, it may print the correct total.** With `-O2` the compiler is allowed to assume no other thread touches `racy` and can transform the loop into a single `racy += 1000000`. The program "works" — by accident. Changing compiler flags, hardware or load can make it fail. That is what *undefined behaviour* looks like in concurrent code.
- **ThreadSanitizer finds it on every run anyway**, because it detects the *missing synchronization*, not the wrong result. Here is TSan's (trimmed) report for this exact program:

ThreadSanitizer output

```
WARNING: ThreadSanitizer: data race (pid=366)
  Write of size 8 at 0x7fff642eae90 by thread T4:
    #0 operator() 04_race.cpp:21
  Previous write of size 8 at 0x7fff642eae90 by thread T3:
    #0 operator() 04_race.cpp:21
  Location is stack of main thread.
```

3.2 Two different things: data race vs race condition

	Data race	Race condition
Definition	Two threads access the **same memory location**, at least one access is a **write**, at least one is **not atomic**, and nothing orders them (no happens-before relation).	The program's **correctness depends on timing** — on which thread gets somewhere first.
Level	Language level: a property of individual memory accesses.	Logic level: a property of your design.
Consequence	**Undefined behaviour.** Anything may happen, including "works on my machine".	Well-defined but wrong results (e.g. double-booking a seat).
Tool support	ThreadSanitizer detects them reliably.	No tool can tell you your logic is wrong. Only design and tests.

You can have a race condition with no data race at all. Every individual call below is protected by a lock, yet the program is still wrong:

```
// Every method of `account` locks internally. No data race...
if (account.balance() >= 100)      // check   (lock taken & released)
    account.withdraw(100);         // act     (lock taken & released)
// ...but two threads can both pass the check, then both withdraw:
// overdraft. This is the "check-then-act" race condition.
```

The cure is to make the check and the action **one indivisible operation** under one lock — e.g. a `bool try_withdraw(amount)` method. We return to this in §10 (thread-safe API design). Other common shapes of race conditions: *read-modify-write* (the counter), *check-then-act* (above, and "if file doesn't exist, create it"), and *publication before initialisation* (another thread sees a pointer to an object before the object is fully constructed).

3.3 Critical sections and invariants

An **invariant** is a statement about your data that must be true whenever anyone looks at it: "`size_` equals the number of elements", "the sum of all account balances is constant", "every node's `next->prev` points back to it". Updating data usually **breaks the invariant temporarily** — in a doubly linked list removal there is a moment when one pointer is updated and the other is not.

A **critical section** is a piece of code during which an invariant may be broken, and which therefore must not be interleaved with any other code that relies on that invariant. Making it *mutually exclusive* — at most one thread inside at a time — is what a mutex is for.

> **KEY IDEA**
>
> Think in terms of **invariants**, not variables. Ask: "What must be true about this data? Which code temporarily breaks it?" Everything that breaks or reads the invariant goes in the same critical section, protected by the same mutex.

Three ways to avoid problems with shared state

1. **Don't share.** Give each thread its own data and combine results at the end (each thread writes its own slot of a vector; see `14_scaling.cpp`). This is the fastest and simplest approach. Prefer it.
2. **Don't mutate.** Data that is fully built before threads start and never changed afterwards (`const`) can be read by any number of threads with no locks.
3. **Synchronize.** If data must be shared *and* mutated, protect it with a mutex (§4), or, for single variables, use an atomic (§9).

> **PITFALL**
>
> `volatile` is **not** a synchronization tool in C++. It does not make operations atomic and does not order memory between threads. It exists for memory-mapped hardware registers. Use `std::atomic` or a mutex.

Exercises §3

1. Compile `04_race.cpp` with `-O0` and `-O2`, with and without `-fsanitize=thread`. Record what you see. Try it on the most-cores machine you can access.
2. Write a two-thread program with a check-then-act race on a `std::vector` ("if empty, push a sentinel"). Make each call locked individually, show the logic is still wrong, then fix it with one combined operation.
3. For a doubly linked list `remove(node)`, write down the invariant, and mark exactly the lines that break it.

4. Mutexes and Locks

4.1 std::mutex

A **mutex** (mutual exclusion) is an object with two operations: `lock()` blocks until the calling thread owns the mutex, `unlock()` releases it. At most one thread owns a `std::mutex` at a time. `try_lock()` acquires it only if that is possible without waiting and returns `true`/`false`.

A mutex does two jobs, and both matter:

- **Mutual exclusion:** critical sections protected by the same mutex never overlap.
- **Visibility (memory ordering):** everything a thread wrote before `unlock()` is guaranteed visible to the next thread after its `lock()` of that mutex. Unlocking is a *release* and locking is an *acquire* (§9). This is why you don't also need atomics for data protected by a mutex.

> **PITFALL**
>
> Never call `lock()`/`unlock()` by hand. Any early `return`, `break`, or exception between them leaves the mutex locked forever and every other thread blocks on it. Always use an RAII lock object.

4.2 The RAII lock types

05_locks.cpp

```cpp
// 05_locks.cpp  --  lock_guard, unique_lock, scoped_lock, shared_mutex
#include <chrono>
#include <iostream>
#include <map>
#include <mutex>
#include <optional>
#include <shared_mutex>
#include <string>
#include <thread>
#include <vector>
// A cache that is read often and written rarely: reader/writer lock.
class PhoneBook {
public:
    std::optional<std::string> find(const std::string& name) const {
        std::shared_lock lk(m_);              // many readers at once
        auto it = book_.find(name);
        if (it == book_.end()) return std::nullopt;
        return it->second;                    // return a COPY, never a ref
    }
    void set(const std::string& name, std::string number) {
        std::unique_lock lk(m_);              // one writer, no readers
        book_[name] = std::move(number);
    }
private:
    mutable std::shared_mutex m_;   // mutable: locked inside const find()
    std::map<std::string, std::string> book_;
};
int main() {
    std::mutex m;
    int shared = 0;
    // lock_guard: lock in constructor, unlock in destructor. Nothing else.
    {
        std::lock_guard<std::mutex> lk(m);
        ++shared;
    }   // unlocked here, even if ++shared had thrown
    // unique_lock: same RAII idea, but flexible (and slightly heavier)
    {
        std::unique_lock<std::mutex> lk(m);
        ++shared;
        lk.unlock();                  // release early ...
        // ... do slow work that doesn't touch `shared` ...
        lk.lock();                    // ... and re-acquire
        ++shared;
    }
    {   // deferred: create unlocked, lock later
        std::unique_lock<std::mutex> lk(m, std::defer_lock);
        lk.lock();
    }
    {   // try_to_lock: don't block if someone else holds it
        std::unique_lock<std::mutex> lk(m, std::try_to_lock);
        if (lk.owns_lock()) std::cout << "got the lock\n";
        else                std::cout << "busy, doing something else\n";
    }
    // scoped_lock (C++17): locks SEVERAL mutexes deadlock-free
    std::mutex a, b;
    {
        std::scoped_lock lk(a, b);
        // both held here
    }
    PhoneBook pb;
    std::thread w([&] { pb.set("alice", "555-0100"); });
    std::vector<std::thread> readers;
    for (int i = 0; i < 4; ++i)
        readers.emplace_back([&] {
            for (int k = 0; k < 1000; ++k) (void)pb.find("alice");
        });
```

```
    w.join();
    for (auto& r : readers) r.join();
    std::cout << "shared = " << shared << ", alice = "
              << pb.find("alice").value_or("?") << "\n";
}
```

Type	Use when	Notes
`std::lock_guard<M>`	You need to lock one mutex for a whole scope. **Default choice.**	Not movable; can't unlock early. Zero overhead over manual lock/unlock.
`std::scoped_lock<M...>` (C++17)	You need to lock **several** mutexes at once, or just one.	Uses a deadlock-avoidance algorithm for multiple mutexes. Can replace `lock_guard` everywhere.
`std::unique_lock<M>`	You need flexibility: unlock early, relock, defer locking, try-lock, timed lock, move ownership — **and always for condition variables**.	Stores an "owns lock" flag, so slightly larger. Movable.
`std::shared_lock<M>` (C++14)	Readers of a `std::shared_mutex`.	Many shared owners at once; excludes exclusive owners.

unique_lock options

Constructor tag	Meaning
(none)	Lock now, blocking.
`std::defer_lock`	Associate with the mutex but don't lock yet. Lock later with `lk.lock()` or `std::lock(lk1, lk2)`.
`std::try_to_lock`	Try once; check `lk.owns_lock()`.
`std::adopt_lock`	The calling thread already locked it; just take over responsibility for unlocking.
duration / time point	With `std::timed_mutex`: give up after a timeout (`try_lock_for` / `try_lock_until`).

4.3 Other mutex types

Mutex	Purpose and caveat
`std::recursive_mutex`	The same thread may lock it several times (must unlock as many times). Usually a sign of unclear design: a public method calling another public method while holding the lock. Prefer splitting into a public locking method and a private non-locking helper.
`std::timed_mutex`	Adds `try_lock_for(duration)` and `try_lock_until(time_point)`.
`std::shared_mutex` (C++17)	Reader–writer lock: many readers **or** one writer. Only pays off when reads greatly outnumber writes *and* reads are not trivially short; otherwise its extra bookkeeping makes it slower than a plain mutex. Measure.

4.4 Using mutexes well

Keep the mutex next to the data it protects

Put the mutex and the data it guards as private members of the same class, and make every public method lock as needed. Then it is impossible to touch the data without the lock. A comment like `// guarded by m_` on each protected member is cheap documentation.

Never let a reference to protected data escape

```
class Registry {
    std::mutex m_;
    std::vector<std::string> names_;
public:
    // BAD: caller gets a reference and uses it AFTER the lock is released.
    const std::vector<std::string>& names() {
        std::lock_guard lk(m_);
        return names_;              // lock released on return!
    }
    // BAD: user callback runs while we hold the lock; it can stash
    // a pointer to names_ or call back into us (self-deadlock).
    template <class F> void with_names(F f) {
        std::lock_guard lk(m_);
        f(names_);
    }
    // GOOD: return a copy (snapshot) made under the lock.
    std::vector<std::string> snapshot() {
        std::lock_guard lk(m_);
        return names_;
    }
};
```

Lock granularity: hold locks briefly

A lock serializes everything it covers. Holding it during I/O, sleeping, or heavy computation turns your parallel program into a serial one. Do the slow work *outside* the lock: copy inputs out, compute, then lock again briefly to publish the result.

```
Item item;
{
    std::lock_guard lk(m_);
    item = queue_.front(); queue_.pop();      // tiny critical section
}
Result r = expensive_compute(item);           // no lock held
{
    std::lock_guard lk(m_);
    results_.push_back(std::move(r));          // tiny critical section
}
```

But don’t go too fine-grained either: if an invariant spans two pieces of data, both must be updated under the same lock, or other threads can observe the broken state between the two updates. Granularity is a trade-off between **correctness** (coarse is simpler) and **contention** (fine is faster).

> **RULE OF THUMB**
>
> **One mutex per invariant.** Lock for as short as possible, but never shorter than the time the invariant is broken.

Exercises §4

1. Wrap a `std::unordered_map<std::string,int>` word counter in a class with `void add(std::string_view)` and `int get(std::string_view) const`. Use it from 4 threads counting words from different files. (Hint: why must the mutex be `mutable`?)
2. Turn the counter into a reader-writer version with `std::shared_mutex`. Benchmark both with 95% reads and 50% reads. Which wins?
3. Rewrite a function that uses manual `lock()`/`unlock()` with two early returns so it uses `std::lock_guard`.

4. Find and fix the lock-escape bug in the `Registry` example above, and explain why `with_names` is dangerous.

5. Deadlocks (and Their Cousins)

5.1 What a deadlock is

A **deadlock** is a state where a set of threads each wait for a resource held by another thread in the set, so none can ever proceed. The program doesn't crash — it just hangs, often only under load, often only in production.

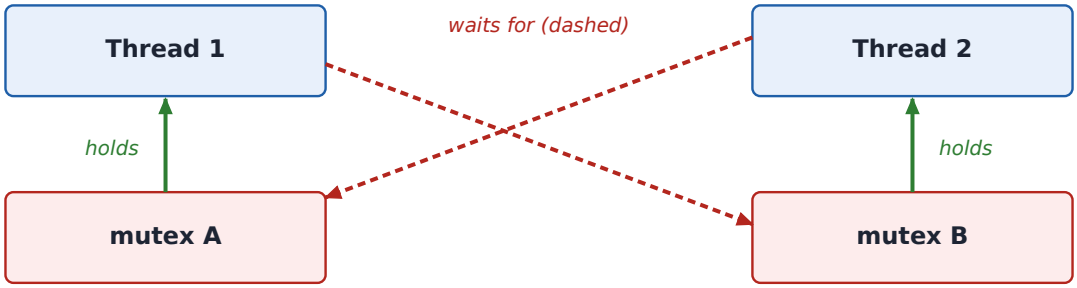


Figure 2. A wait-for cycle: T1 holds A and waits for B; T2 holds B and waits for A. Neither can move.

5.2 The four necessary conditions (Coffman, 1971)

A deadlock can occur only if **all four** of these hold at once. Break any one and deadlock is impossible:

Condition	Meaning	How to break it
Mutual exclusion	Resources can be held by only one thread.	Use lock-free / immutable data (rarely practical).
Hold and wait	A thread holds one resource while waiting for another.	Acquire everything at once (`std::scoped_lock`), or never hold a lock while acquiring another.
No preemption	Resources can't be taken away; only released voluntarily.	Use `try_lock`: on failure release what you hold and retry.
Circular wait	A cycle exists in the wait-for graph.	**Global lock ordering**: always acquire in one agreed order.

5.3 The canonical example and its fix

06_deadlock.cpp

```cpp
// 06_deadlock.cpp  --  the classic two-account transfer
#include <iostream>
#include <mutex>
#include <thread>
struct Account {
    std::mutex m;
    long balance = 1000;
};
// DEADLOCK-PRONE version (shown for study; not called):
// T1: transfer_bad(a, b)  locks a, then waits for b
// T2: transfer_bad(b, a)  locks b, then waits for a   -> both stuck forever
void transfer_bad(Account& from, Account& to, long amt) {
    std::lock_guard<std::mutex> l1(from.m);
    std::lock_guard<std::mutex> l2(to.m);
    from.balance -= amt;
    to.balance   += amt;
}
// SAFE: std::scoped_lock acquires both using a deadlock-avoidance
// algorithm, regardless of the order the arguments are passed in.
void transfer(Account& from, Account& to, long amt) {
    if (&from == &to) return;     // locking the same mutex twice is UB
    std::scoped_lock lk(from.m, to.m);
    from.balance -= amt;
    to.balance   += amt;
}
int main() {
    Account a, b;
    std::thread t1([&] { for (int i = 0; i < 100000; ++i) transfer(a, b, 1); });
    std::thread t2([&] { for (int i = 0; i < 100000; ++i) transfer(b, a, 1); });
    t1.join(); t2.join();
    std::cout << a.balance << " + " << b.balance << " = "
              << a.balance + b.balance << "\n";   // always 2000
}
```

`transfer_bad(a, b)` and `transfer_bad(b, a)` lock in opposite orders. Run them concurrently and sooner or later each thread holds one account and waits for the other. `std::scoped_lock` (or `std::lock` + `adopt_lock` before C++17) locks both mutexes **as one operation** with a back-off algorithm, so argument order no longer matters.

5.4 Prevention techniques

1. **Lock several mutexes together** with `std::scoped_lock(m1, m2, ...)`.
2. **Impose a global order.** When you can't lock together (locks taken in different functions), document an order and always follow it — e.g. by address, by account id, or by **lock hierarchy** level: a thread holding a level-5 lock may only acquire locks of level < 5. You can even enforce the hierarchy at run time with a wrapper mutex that checks a `thread_local` "current level" (Williams, ch. 3.2.5).
3. **Avoid nested locks.** If a function already holds a lock, don't acquire another unless you must.
4. **Never call unknown code while holding a lock** — callbacks, virtual functions, user-supplied lambdas. They might take locks you don't know about.
5. **Don't wait for a thread while holding a lock it needs.** `join()`ing a thread under a mutex that the thread needs to finish is a deadlock with only one mutex.
6. **Prefer timeouts in long-lived systems** (`try_lock_for`) so a stuck lock produces a log message instead of a silent hang.

PITFALL

Self-deadlock: locking a non-recursive `std::mutex` that the same thread already holds is undefined behaviour — in practice it hangs forever. It typically happens when public method `a()` locks `m_` and calls public method `b()`, which also locks `m_`.

5.5 Livelock and starvation

- **Livelock:** threads are not blocked but keep reacting to each other without progressing — e.g. two threads that each `try_lock` the second mutex, fail, release the first, and retry in perfect lockstep forever. Fixed with randomised back-off.
- **Starvation:** some thread never gets its turn, e.g. a writer that never acquires a `shared_mutex` because readers keep arriving. Whether `std::shared_mutex` prevents writer starvation is implementation-defined.
- **Priority inversion:** a high-priority thread waits for a lock held by a low-priority thread that never gets scheduled. Famous from the Mars Pathfinder mission; mainly an OS / real-time concern.

5.6 Debugging a hang

Attach a debugger to the stuck process and look at every thread's stack. Deadlocked threads sit in `pthread_mutex_lock` / `__lll_lock_wait` (Linux) with two frames above that showing which of *your* locks they are waiting for.

```
$ gdb -p <pid>            # or: lldb -p <pid>
(gdb) info threads         # list all threads
(gdb) thread apply all bt  # backtrace of every thread  (lldb: bt all)
```

ThreadSanitizer also reports **lock-order inversions** ("potential deadlock") even on runs that happened not to hang, because it records the order in which every pair of mutexes is acquired.

SYSTEMS CONNECTION

The OS course studies the same four conditions for resources in general (memory, devices, files), plus detection via resource-allocation graphs and avoidance via the Banker's algorithm. In distributed systems the same cycles appear between processes waiting on each other's messages or locks.

Exercises §5

1. Make `transfer_bad` deadlock on purpose (add `std::this_thread::sleep_for(1ms)` between the two locks to widen the window). Attach gdb and find the cycle in the backtraces.
2. Fix it three ways: `std::scoped_lock`; ordering by account address; ordering by an `id` member. Which is most robust?
3. Implement a `hierarchical_mutex` with a level number that throws if a thread tries to lock a level ≥ the level it currently holds. (Williams 2e, section 3.2.5 has a reference version to compare against *after* you try.)

6. Condition Variables

6.1 The problem: waiting for something to become true

Mutexes protect data, but often a thread needs to **wait until some condition holds**: "the queue is non-empty", "the result is ready", "shutdown was requested". Two bad solutions:

```
while (!ready) {}                       // 1. busy-wait: burns a core at 100%,
                                        //    AND is a data race if ready isn't atomic
while (true) {                          // 2. poll with sleep: slow to react,
    { std::lock_guard lk(m); if (ready) break; }   //    still wastes wakeups
    std::this_thread::sleep_for(10ms);
}
```

A **condition variable** lets a thread go to sleep *atomically releasing the mutex*, and be woken by another thread when the state may have changed. It is always used together with (1) a mutex and (2) a piece of shared state — the **predicate** — protected by that mutex. The CV itself holds no state; it is only a notification channel.

6.2 The canonical pattern

07_cv_handshake.cpp

```
// 07_cv_handshake.cpp  --  one thread waits for another to be ready
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <string>
#include <thread>
std::mutex m;
std::condition_variable cv;
bool ready = false;          // the CONDITION. Protected by m.
std::string payload;
void consumer() {
    std::unique_lock<std::mutex> lk(m);   // wait() needs a unique_lock
    cv.wait(lk, [] { return ready; });    // sleeps; re-checks on wakeup
    // Here: lock is held AND ready == true.
    std::cout << "consumer got: " << payload << "\n";
}
void producer() {
    {
        std::lock_guard<std::mutex> lk(m);
        payload = "the data";
        ready = true;                     // change state UNDER the lock
    }
    cv.notify_one();                      // then wake the waiter
}
int main() {
    std::thread c(consumer), p(producer);
    p.join(); c.join();
}
```

`cv.wait(lk, pred)` is exactly equivalent to:

```
while (!pred()) {        // check the condition with the lock held
    cv.wait(lk);         // atomically: unlock m + sleep; on wakeup: relock m
}
```

Walk through what happens:

1. The consumer locks `m`, checks `ready` (false), and calls `wait`. `wait` **releases `m` and blocks** in one atomic step — there is no window where the producer could notify between the

unlock and the sleep.

2. The producer locks `m` (now free), sets `ready = true`, unlocks, and calls `notify_one()`.
3. The consumer wakes up, **re-acquires** `m` inside `wait`, re-checks `ready` (true), and returns with the lock held.

6.3 The rules — and why each one exists

Rule	Why
Always wait with a predicate (or in a `while` loop).	**Spurious wakeups:** `wait` may return without anyone notifying (permitted by the standard; happens in real implementations). Also, by the time a woken thread re-acquires the lock, another thread may already have consumed the item ("stolen wakeup"). Re-checking handles both.
Modify the predicate's state while holding the same mutex, even if the variable is atomic.	**Lost wakeup:** if the producer sets `ready` without the lock, it can happen *between* the consumer's check and its sleep. The notify arrives while nobody is waiting, and the consumer then sleeps forever.
Use `std::unique_lock`, not `lock_guard`.	`wait` must unlock and relock the mutex; only `unique_lock` allows that.
A notification is not stored.	If nobody is waiting, `notify_*` does nothing. The *state* (the predicate) is what carries the information; the notify only says "go look again". That's why you never wait without checking the state first.

6.4 notify_one vs notify_all; notify inside or outside the lock

- `notify_one()` wakes one waiter. Use it when any single waiter can handle the event (one item was pushed into a queue).
- `notify_all()` wakes every waiter. Use it when the change matters to all of them (shutdown; a "go" signal; when waiters are waiting for *different* predicates on the same CV). When in doubt, `notify_all` is correct but may be slower.
- Notifying **after** unlocking is legal and slightly more efficient: the woken thread doesn't immediately block on the mutex you still hold. Notifying while holding the lock is also correct. What matters is that the *state change* happened under the lock.

6.5 Timed waits

```
std::unique_lock lk(m);
using namespace std::chrono_literals;
if (cv.wait_for(lk, 500ms, [&] { return ready; })) {
    // predicate became true in time (lock held)
} else {
    // timed out and predicate still false (lock held)
}
// wait_until(lk, deadline, pred) takes an absolute time point instead.
```

RULE OF THUMB

The three-part recipe: **(1) a mutex, (2) state protected by it, (3) a CV.** Waiters: lock → `wait(lk, predicate)` → act. Notifiers: lock → change state → unlock → notify. If any part is missing, it's wrong.

`std::condition_variable_any` works with any lockable type (e.g. `std::shared_lock`) and in C++20 can wait with a `std::stop_token`, waking automatically on `request_stop()`. It is slightly slower than `std::condition_variable`.

Exercises §6

1. Remove the predicate from `07_cv_handshake.cpp` (call plain `cv.wait(lk)`) and start the producer *first*. What can happen? Why does the predicate version not have this problem?
2. Write a “ping-pong” program: two threads alternately print `ping` and `pong` 10 times each, using one mutex, one CV and a `turn` variable.
3. Implement a `CountDown` class with `count_down()` and `wait()` (that is what C++20 `std::latch` is), using a mutex and a CV.

7. The Producer–Consumer Pattern

7.1 Idea

Producers create work items; **consumers** process them; a **queue** in between decouples them so each side can run at its own pace. It is the single most important concurrency pattern: thread pools, logging systems, network servers, GPU command queues, OS I/O schedulers and message brokers like Kafka are all producer–consumer systems. Your Week 6 capstone pipeline is a chain of them.

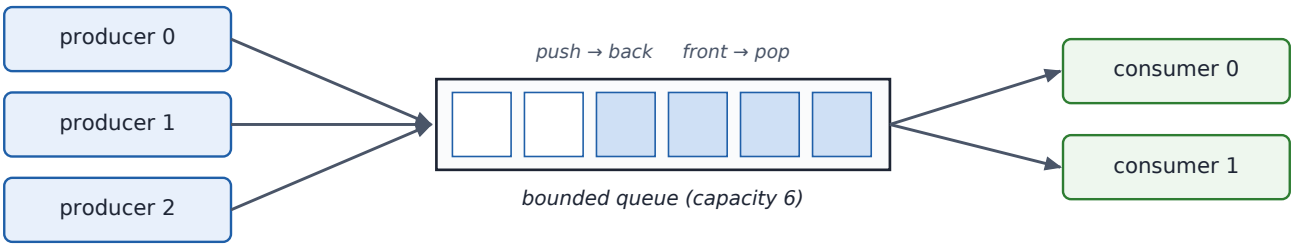


Figure 3. Producers block when the queue is full (back-pressure); consumers block when it is empty.

7.2 Why bounded?

An **unbounded** queue lets fast producers allocate memory until the process dies. A **bounded** queue applies **back-pressure**: when it is full, `push` blocks, which slows producers down to the speed of consumers. So we need two conditions: *not empty* (consumers wait on it) and *not full* (producers wait on it).

7.3 A complete, shutdown-aware implementation

08_bounded_queue.cpp

```cpp
// 08_bounded_queue.cpp  --  multi-producer / multi-consumer queue
#include <condition_variable>
#include <cstddef>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <thread>
#include <vector>
template <class T>
class BoundedQueue {
public:
    explicit BoundedQueue(std::size_t capacity) : cap_(capacity) {}
    // Blocks while full. Returns false if the queue was closed.
    bool push(T value) {
        std::unique_lock lk(m_);
        not_full_.wait(lk, [&] { return q_.size() < cap_ || closed_; });
        if (closed_) return false;
        q_.push_back(std::move(value));
        lk.unlock();                 // optional: wake the waiter without
        not_empty_.notify_one();     // it immediately blocking on m_
        return true;
    }
    // Blocks while empty. Returns nullopt once closed AND drained.
    std::optional<T> pop() {
        std::unique_lock lk(m_);
        not_empty_.wait(lk, [&] { return !q_.empty() || closed_; });
        if (q_.empty()) return std::nullopt;     // closed and drained
        T v = std::move(q_.front());
        q_.pop_front();
        lk.unlock();
        not_full_.notify_one();
        return v;
    }
    // No more pushes. Wake EVERYONE so blocked threads can observe it.
    void close() {
        { std::lock_guard lk(m_); closed_ = true; }
        not_empty_.notify_all();
        not_full_.notify_all();
    }
private:
    std::mutex m_;
    std::condition_variable not_empty_, not_full_;
    std::deque<T> q_;
    std::size_t cap_;
    bool closed_ = false;
};
int main() {
    BoundedQueue<int> q(8);
    constexpr int kProducers = 3, kConsumers = 2, kPer = 10000;
    std::vector<std::thread> producers, consumers;
    std::vector<long> sums(kConsumers, 0);   // one slot per consumer
    for (int p = 0; p < kProducers; ++p)
        producers.emplace_back([&] {
            for (int i = 1; i <= kPer; ++i) q.push(i);
        });
    for (int c = 0; c < kConsumers; ++c)
        consumers.emplace_back([&, c] {
            while (auto v = q.pop()) sums[c] += *v;  // exits on nullopt
        });
    for (auto& t : producers) t.join();
    q.close();                               // producers are done
    for (auto& t : consumers) t.join();
    long total = 0;
    for (long s : sums) total += s;
    long expected = long(kProducers) * kPer * (kPer + 1) / 2;
```

```
    std::cout << "total " << total << " expected " << expected << "\n";
}
```

7.4 Design decisions worth understanding

- **Two condition variables.** Producers and consumers wait for different predicates, so they get separate CVs and `notify_one` always wakes the right kind of thread. With one CV you'd need `notify_all` to be safe.
- `pop()` **returns** `std::optional<T>` — it fuses "check empty", "read front" and "remove" into one operation (see §10) and gives a natural "no more items" signal.
- `close()` **is the shutdown protocol.** Without it, consumers blocked in `pop()` on an empty queue would wait forever and you could never join them. `close()` sets a flag under the lock and `notify_all`s *both* CVs so every sleeper re-checks its predicate. Consumers keep draining remaining items and only stop when the queue is closed **and** empty.
- **Results go into per-consumer slots** (`sums[c]`) instead of one shared total, so consumers never contend on a result lock.
- Alternative shutdown: push one special **poison pill** item per consumer. Works, but `close()` is cleaner and doesn't require knowing how many consumers exist.

> **SYSTEMS CONNECTION**
>
> The OS course solves the same bounded-buffer problem with two **semaphores** (`empty`, `full`) and a mutex (OSTEP ch. 31). C++20 has `std::counting_semaphore` too. Condition variables are more general because the predicate can be anything, not just a count.

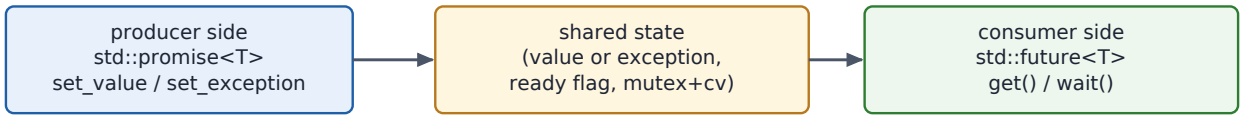
Exercises §7

1. Re-type `BoundedQueue` from memory, without looking. Then diff it against the listing.
2. Add `bool try_push(T)` and `std::optional<T> try_pop()` (non-blocking) and `pop_for(duration)` (timed).
3. Set capacity to 1 and to 1000; time the program for each. Explain the difference.
4. Turn it into a two-stage pipeline: stage 1 reads lines from a file and pushes them; stage 2 workers count words and push counts into a second queue; stage 3 aggregates. Shut it down cleanly. (This is a mini-preview of Week 6.)

8. Futures, Promises, packaged_task and async

8.1 The idea: a one-shot channel for a result

Threads give you no way to return a value. **Futures** fix that. A *future* is the **receiving end** of a one-shot channel that will eventually hold either a **value** or an **exception**. The **sending end** is a *promise* (or something built on one, like `packaged_task` or `async`). Together they share a hidden heap-allocated **shared state**.



One value, delivered once. get() blocks until ready; exceptions are rethrown in the caller.

Figure 4. promise → shared state → future.

09_futures.cpp

```cpp
// 09_futures.cpp  --  async, promise/future, packaged_task
#include <future>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <thread>
#include <vector>
long sum_range(const std::vector<int>& v, size_t lo, size_t hi) {
    return std::accumulate(v.begin() + lo, v.begin() + hi, 0L);
}
int main() {
    std::vector<int> data(1'000'000, 1);
    // --- std::async: run a function, get a future for its result -------
    auto half = data.size() / 2;
    std::future<long> f = std::async(std::launch::async, sum_range,
                                     std::cref(data), 0, half);
    long second = sum_range(data, half, data.size());  // main does half
    std::cout << "async sum = " << f.get() + second << "\n";
    // --- exceptions travel through the future -------------------------
    auto bad = std::async(std::launch::async, []() -> int {
        throw std::runtime_error("worker failed");
    });
    try { bad.get(); }
    catch (const std::exception& e) {
        std::cout << "caught: " << e.what() << "\n";
    }
    // --- std::promise: a thread fulfils a value "by hand" --------------
    std::promise<int> prom;
    std::future<int> fut = prom.get_future();
    std::thread t([p = std::move(prom)]() mutable {
        p.set_value(42);        // or p.set_exception(...)
    });
    std::cout << "promise gave " << fut.get() << "\n";
    t.join();
    // --- std::packaged_task: wrap a callable, run it anywhere later ----
    std::packaged_task<int(int, int)> task([](int a, int b) { return a * b; });
    std::future<int> tf = task.get_future();
    std::thread t2(std::move(task), 6, 7);   // task runs on t2
    std::cout << "packaged_task gave " << tf.get() << "\n";
    t2.join();
    // --- shared_future: many threads wait on ONE result ----------------
    std::promise<void> go;
```

```
    std::shared_future<void> start = go.get_future().share();
    std::vector<std::thread> runners;
    for (int i = 0; i < 3; ++i)
        runners.emplace_back([start, i] {
            start.wait();                    // all block on the same signal
            std::cout << "runner " + std::to_string(i) + " started\n";
        });
    go.set_value();                          // release everyone at once
    for (auto& r : runners) r.join();
}
```

8.2 The four tools, compared

Tool	What you get	Use it for
`std::async(policy, f, args...)`	Runs `f` somewhere and returns a `future`.	Quick fire-and-get-result parallelism.
`std::promise<T>`	A slot you fill by hand with `set_value` / `set_exception`; `get_future()` gives the receiving end.	When the result is produced by code that isn't a single function call (e.g. an event callback, or a value computed half-way through a long-running thread).
`std::packaged_task<R(Args...)>`	Wraps a callable; calling the task runs it and stores the result/exception in its future.	**Task queues and thread pools**: the submitter keeps the future, the worker just calls the task.
`std::shared_future<T>`	A copyable future; many threads can `get()` the same result.	Broadcasting one result or a "start" signal.

8.3 Important details

- `get()` can be called **once** on a `std::future`; afterwards `valid()` is false. Use `shared_future` for multiple readers.
- If a promise is destroyed without setting a value, the future's `get()` throws `std::future_error` with `broken_promise`. So consumers never wait forever on a crashed producer.
- `wait_for(timeout)` returns `std::future_status::ready`, `timeout`, or `deferred`, which lets you poll without blocking.
- `std::future<void>` and `std::promise<void>` carry no value — just "done" (or an exception). Great as one-shot signals.

8.4 std::async pitfalls

PITFALL

Launch policy. With no policy, `std::async(f)` may run `f` on a new thread **or** defer it until you call `get()`/`wait()` — running it on *your* thread. If you need real concurrency, always pass `std::launch::async`.

PITFALL

The blocking destructor. The future returned by `std::async` blocks in its destructor until the task finishes. So `std::async(std::launch::async, f);` — discarding the result — runs **synchronously**: the temporary future is destroyed at the end of the statement and waits. Always keep the future.

`std::async` is fine for a handful of coarse tasks, but GCC's and Clang's standard libraries start a new thread per call (MSVC uses a system pool), and none give you control over the number of threads. That is one motivation for writing your own **thread pool**.

> **SYSTEMS CONNECTION**
>
> Futures/promises are the same idea as JavaScript Promises, Rust futures, and the RPC stubs of distributed systems (call now, get the reply later). In CUDA, streams and events play a similar role for GPU work.

Exercises §8

1. Parallel sum: split a `std::vector<int>` of 100M elements into k chunks, sum each with `std::async(std::launch::async, ...)`, and combine. Time $k = 1, 2, 4, 8$.
2. Write a function that starts a thread which throws halfway through its work. Deliver the exception to `main` with a `std::promise`.
3. Prove the blocking-destructor pitfall: call `std::async(std::launch::async, sleep_1s)` three times without keeping the futures, and time it.
4. Use a `promise<void>` + `shared_future<void>` to start 8 threads at the same instant, then measure the spread of their start times.

9. Atomics and the Memory Model

9.1 What std::atomic gives you

`std::atomic<T>` (header `<atomic>`) makes every operation on a single variable **indivisible**: no other thread can observe it half-done, and concurrent atomic accesses are never a data race. For integral types and pointers, read-modify-write operations like `++` are done as one hardware instruction (e.g. `lock xadd` on x86) instead of load–add–store, so no updates are lost. That is why FIX 2 in `04_race.cpp` works without a mutex.

Operation	Meaning
`load()` / `store(v)` / implicit conversion and `=`	Atomic read / write.
`exchange(v)`	Write `v`, return the old value, atomically.
`fetch_add`, `fetch_sub`, `fetch_and`, `fetch_or`, `fetch_xor`; `++`, `--`, `+=` ...	Atomic read-modify-write; `fetch_*` return the **old** value.
`compare_exchange_weak(expected, desired)` / `_strong`	**CAS**: if current == `expected`, store `desired` and return true; otherwise **load the current value into** `expected` and return false.
`wait(old)`, `notify_one()`, `notify_all()` (C++20)	Block until the value differs from `old` — a lightweight futex-style wait.
`is_lock_free()`	Whether this type is implemented with real atomic instructions rather than a hidden lock.

`std::atomic<T>` works for any trivially copyable `T`, but only small types (typically up to 8 or 16 bytes) are lock-free. `std::atomic_flag` is the one type guaranteed lock-free everywhere. Atomics are **not copyable**.

9.2 Compare-and-swap: the building block of lock-free code

CAS lets you implement *any* read-modify-write on one variable: read the value, compute the new one, and publish it only if nobody changed the variable in between; otherwise retry with the fresh value.

10_atomics.cpp

```cpp
// 10_atomics.cpp  --  atomic operations and memory ordering
#include <atomic>
#include <cassert>
#include <iostream>
#include <string>
#include <thread>
#include <vector>
// ---- 1. compare_exchange: update a running maximum lock-free ----------
void update_max(std::atomic<int>& mx, int candidate) {
    int cur = mx.load();
    // If mx still equals cur, replace it with candidate. Otherwise cur is
    // reloaded with the current value and we try again.
    while (candidate > cur && !mx.compare_exchange_weak(cur, candidate)) {
    }
}
// ---- 2. a minimal spinlock (for learning; prefer std::mutex) ----------
class SpinLock {
    std::atomic_flag f_ = ATOMIC_FLAG_INIT;
public:
    void lock()   { while (f_.test_and_set(std::memory_order_acquire)) {} }
    void unlock() { f_.clear(std::memory_order_release); }
};
```

```
// ---- 3. release/acquire message passing ------------------------------
std::string message;                 // plain, non-atomic data
std::atomic<bool> published{false};
void writer() {
    message = "hello";                                  // (A)
    published.store(true, std::memory_order_release); // (B) publish
}
void reader() {
    while (!published.load(std::memory_order_acquire)) {} // (C)
    assert(message == "hello");      // (D) guaranteed: A happens-before D
}
int main() {
    std::atomic<int> mx{0};
    std::vector<std::thread> ts;
    for (int i = 0; i < 8; ++i)
        ts.emplace_back([&mx, i] {
            for (int k = 0; k < 1000; ++k) update_max(mx, i * 1000 + k);
        });
    for (auto& t : ts) t.join();
    std::cout << "max = " << mx << "\n";               // 7999
    SpinLock sl; long n = 0; ts.clear();
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([&] {
            for (int k = 0; k < 100000; ++k) { sl.lock(); ++n; sl.unlock(); }
        });
    for (auto& t : ts) t.join();
    std::cout << "spin n = " << n << "\n";             // 400000
    std::thread r(reader), w(writer);
    w.join(); r.join();
    std::cout << "message passed safely\n";
    std::atomic<long long> big;
    std::cout << "atomic<long long> lock-free? " << big.is_lock_free() << "\n";
}
```

- `compare_exchange_weak` may fail **spuriously** (return false even though the values were equal) on some CPUs (LL/SC architectures like ARM). It is cheaper in a loop, which you need anyway. Use `_strong` when not looping.
- Notice CAS loads the current value into `cur` on failure — you don't reload it yourself.

PITFALL

Atomic variables are not atomic algorithms. `if (x.load() == 0) x.store(1);` is two atomic operations with a gap between them — a check-then-act race. Combine them into one CAS: `int z = 0; x.compare_exchange_strong(z, 1);`. Likewise, two separate atomics can't maintain an invariant that relates them. Once more than one variable is involved, reach for a mutex.

9.3 Why ordering matters

Atomicity of one variable is not enough. Consider a writer that prepares a message, then sets a flag; a reader waits for the flag, then reads the message (bottom of `10_atomics.cpp`). Without ordering guarantees, the compiler or the CPU could make the flag store visible **before** the message store, and the reader would see the flag set and read a half-written message. The **memory order** argument to each atomic operation controls which other memory accesses can be reordered around it.

9.4 The three orderings you need

Order	Guarantee	Typical use
`memory_order_seq_cst` (the **default**)	All seq_cst operations appear in **one global order** that every thread agrees on, consistent with program order. Easiest to reason about.	Everything, unless profiling proves it matters.
`memory_order_release` (on a store) + `memory_order_acquire` (on a load)	If the acquire-load reads the value written by the release-store, then **everything the writer did before the store is visible to the reader after the load**. This creates a *happens-before* edge.	Publishing data, flags, lock implementations, queues.
`memory_order_relaxed`	Only atomicity of this one variable. **No** ordering of other memory around it.	Pure counters / statistics where nobody infers anything from the value.

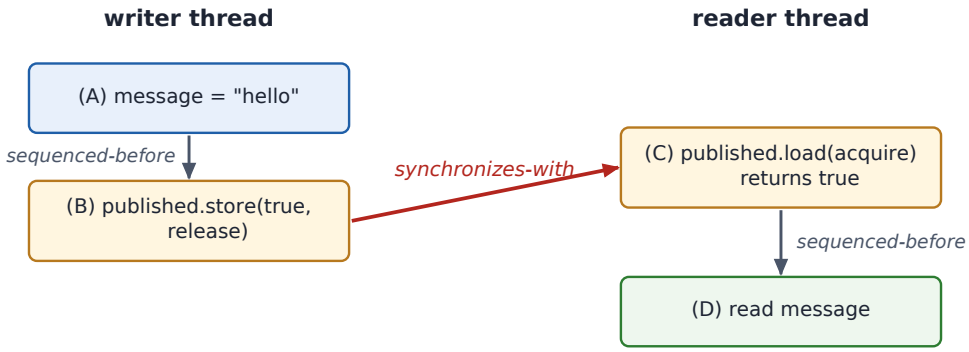


Figure 5. A → B (program order), B synchronizes-with C (release read by acquire), C → D. So A happens-before D: the reader is guaranteed to see "hello".

KEY IDEA

The pairing is the point: **release publishes, acquire subscribes.** A release store with no matching acquire load, or an acquire load that reads a value not written by a release store, gives no ordering. Mutexes use exactly this internally: `unlock()` is a release and `lock()` is an acquire.

Our spinlock uses the same pattern: `test_and_set(acquire)` in `lock()` and `clear(release)` in `unlock()`, so everything done inside one critical section is visible to the next thread that takes the lock.

RULE OF THUMB

For this course: use the default `seq_cst` everywhere, use `relaxed` only for standalone counters, and use acquire/release only when you can draw the happens-before arrow like Figure 5. Lock-free data structures beyond a counter or a flag are an advanced topic — use a mutex, measure, and only then consider going lock-free.

SYSTEMS CONNECTION

x86 is a strongly ordered CPU (TSO), so many ordering mistakes don't show up on your laptop and then break on ARM (phones, Apple Silicon, many servers). GPUs and CUDA have an even weaker memory model, with explicit scopes (`__threadfence`, `cuda::atomic` with thread_scope). The C++ model is your portable contract.

9.5 Spinlocks vs mutexes

A spinlock waits by looping on an atomic, never sleeping. It is great when critical sections are a few instructions long *and* every thread has its own core. It is terrible otherwise: a spinning thread burns CPU, and if the lock holder is preempted, spinners waste their entire time slice. `std::mutex` on Linux spins briefly and then sleeps in the kernel (a *futex*), getting the best of both. **Use `std::mutex` by default.**

9.6 False sharing: a performance bug with no wrong answer

CPUs move memory between cores in **cache lines** (usually 64 bytes). If two threads write to *different* variables that happen to sit in the *same* cache line, the coherence protocol bounces the line between their cores on every write. The program is correct but can be several times slower than expected.

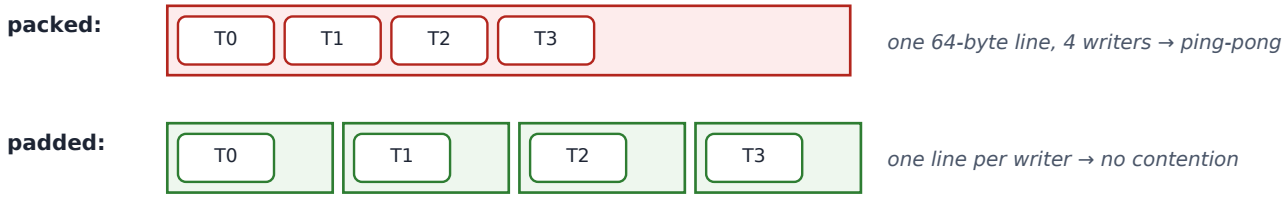


Figure 6. False sharing. Padding each counter to its own cache line removes the contention.

11_false_sharing.cpp

```cpp
// 11_false_sharing.cpp  --  same work, very different speed
#include <atomic>
#include <chrono>
#include <iostream>
#include <thread>
#include <vector>
constexpr int kThreads = 4;
constexpr long kIters  = 20'000'000;
struct Packed { std::atomic<long> v{0}; };              // 8 bytes each
struct alignas(64) Padded { std::atomic<long> v{0}; };  // own cache line
template <class Slot>
double run() {
    std::vector<Slot> slots(kThreads);   // each thread owns ONE slot
    auto t0 = std::chrono::steady_clock::now();
    std::vector<std::thread> ts;
    for (int i = 0; i < kThreads; ++i)
        ts.emplace_back([&slots, i] {
            for (long k = 0; k < kIters; ++k)
                slots[i].v.fetch_add(1, std::memory_order_relaxed);
        });
    for (auto& t : ts) t.join();
    std::chrono::duration<double> d = std::chrono::steady_clock::now() - t0;
    return d.count();
}
int main() {
    std::cout << "packed (false sharing): " << run<Packed>() << " s\n";
    std::cout << "padded (64B apart)    : " << run<Padded>() << " s\n";
}
```

`alignas(64)` puts each counter on its own line. C++17 offers `std::hardware_destructive_interference_size` in `<new>` as a portable constant, though some compilers warn about using it in ABI-relevant places. On a multi-core machine the padded version is typically much faster. (On a single-core machine both take the same time — there is nobody to bounce the line to.)

10. Designing Thread-Safe APIs

10.1 Thread-safe implementation ≠ thread-safe interface

Locking every method makes each *call* safe, but callers combine calls, and races live **between** calls. `std::stack`'s interface (`empty()`, `top()`, `pop()`) is a textbook example: it is designed for single-threaded use, where splitting them is convenient and exception-safe. Under concurrency the split is fatal.

12_ts_stack.cpp

```cpp
// 12_ts_stack.cpp  --  designing an interface that is safe to USE
#include <iostream>
#include <mutex>
#include <optional>
#include <stack>
#include <thread>
#include <vector>
// Every method is locked, yet this API is still racy for callers:
//   if (!s.empty()) { auto x = s.top(); s.pop(); }
// Between empty(), top() and pop() another thread can pop the element.
// The race is in the INTERFACE, not in the implementation.
template <class T>
class ThreadSafeStack {
public:
    void push(T v) {
        std::lock_guard lk(m_);
        s_.push(std::move(v));
    }
    // check + read + remove fused into ONE atomic operation
    std::optional<T> try_pop() {
        std::lock_guard lk(m_);
        if (s_.empty()) return std::nullopt;
        T v = std::move(s_.top());
        s_.pop();
        return v;
    }
    // size() is only a snapshot: it may be stale the moment it returns.
    std::size_t size() const {
        std::lock_guard lk(m_);
        return s_.size();
    }
private:
    mutable std::mutex m_;
    std::stack<T> s_;
};
int main() {
    ThreadSafeStack<int> s;
    for (int i = 0; i < 1000; ++i) s.push(i);
    std::vector<std::thread> ts;
    std::vector<int> popped(4, 0);
    for (int t = 0; t < 4; ++t)
        ts.emplace_back([&, t] { while (s.try_pop()) ++popped[t]; });
    for (auto& t : ts) t.join();
    int total = 0;
    for (int p : popped) total += p;
    std::cout << "popped " << total << " items exactly once each\n";
}
```

10.2 Guidelines

1. **Make each operation a complete, meaningful transaction.** Fuse check + act: `try_pop()`, `try_withdraw()`, `insert_if_absent()`, `get_or_compute()`, `compare_exchange`.
2. **Return by value, not by reference or pointer to internals.** A reference outlives the lock. Return copies, `std::optional<T>`, or move the element out.
3. **Treat queries as snapshots.** `size()`, `empty()`, `contains()` are stale the moment they return. Fine for logging/metrics; never use them to decide what to do next.
4. **Don't take callbacks under the lock** (see §5.4), or document very clearly that you do.
5. **Encapsulate the mutex.** Callers should not have to know a lock exists. If they must lock externally, the class isn't thread-safe.
6. **Document the guarantee.** Is it "safe to call any method concurrently"? "safe for concurrent reads only"? "not thread-safe; confine to one thread"? Say which.
7. **Make blocking explicit and stoppable.** Blocking calls (`pop`, `wait`) need a way out: `close()`, a stop token, or a timeout variant.

The standard library's own guarantee

Standard library types follow a simple convention: concurrent calls to `const` member functions on the same object are safe; any concurrent call involving a non-const member function needs external synchronization. So two threads may call `v.size()` and `v[3]` on a `const std::vector` concurrently, but not `push_back` while another reads. Make your own classes obey the same rule: if a `const` method mutates internal state (a cache, a mutex), that state must be `mutable` *and* synchronized.

10.3 Safe one-time initialization and per-thread state

13_once_tls.cpp

```cpp
// 13_once_tls.cpp  --  one-time init and per-thread state
#include <iostream>
#include <mutex>
#include <string>
#include <thread>
#include <vector>
struct Config { std::string name = "prod"; };
// Option A: function-local static. Initialisation is thread-safe
// since C++11 ("magic statics"). Simplest choice; prefer it.
Config& config() {
    static Config c;          // constructed exactly once
    return c;
}
// Option B: std::call_once, for when init isn't a static's constructor.
std::once_flag init_flag;
void init_logging() { std::cout << "logging initialised once\n"; }
// thread_local: every thread gets its own independent copy.
thread_local int calls_on_this_thread = 0;
int main() {
    std::vector<std::thread> ts;
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([i] {
            std::call_once(init_flag, init_logging);
            (void)config();
            for (int k = 0; k <= i; ++k) ++calls_on_this_thread;
            std::cout << "thread " + std::to_string(i) + " made " +
                         std::to_string(calls_on_this_thread) + " calls\n";
        });
    for (auto& t : ts) t.join();
}
```

- **Function-local statics** are initialized exactly once, thread-safely (C++11). The cleanest way to build a lazy singleton.
- `std::call_once` + `std::once_flag` runs a function exactly once even if many threads race to call it. If it throws, the next caller tries again.
- `thread_local` variables have one instance per thread, created on first use in that thread. Useful for per-thread caches, random number generators, and scratch buffers — a form of "don't share".

PITFALL

Double-checked locking written by hand with a plain pointer (`if (!p) { lock; if (!p) p = new T; }`) is a data race and is broken. Use a function-local static, `std::call_once`, or an atomic pointer with acquire/release.

10.4 Other C++20 coordination tools (beyond the syllabus, good to know)

15_latch_barrier.cpp

```cpp
// 15_latch_barrier.cpp  --  C++20 coordination primitives
#include <barrier>
#include <iostream>
#include <latch>
#include <string>
#include <thread>
#include <vector>
int main() {
    constexpr int N = 3;
    // latch: a one-shot countdown. Workers count down; main waits for 0.
    std::latch all_loaded(N);
    std::vector<std::jthread> loaders;
    for (int i = 0; i < N; ++i)
        loaders.emplace_back([&, i] {
            std::cout << "loader " + std::to_string(i) + " done\n";
            all_loaded.count_down();
        });
    all_loaded.wait();
    std::cout << "all loaded -- main continues\n";
    // barrier: reusable. Every phase, all N threads meet before going on.
    std::barrier sync(N, []() noexcept { std::cout << "-- phase end --\n"; });
    std::vector<std::jthread> workers;
    for (int i = 0; i < N; ++i)
        workers.emplace_back([&, i] {
            for (int phase = 0; phase < 2; ++phase) {
                std::cout << "w" + std::to_string(i) + " phase " +
                             std::to_string(phase) + "\n";
                sync.arrive_and_wait();
            }
        });
}
```

Tool	What it does
`std::latch`	One-shot countdown: threads `count_down()`, others `wait()` until zero. Can't be reused.
`std::barrier`	Reusable rendezvous: all N threads `arrive_and_wait()`; an optional completion function runs once per phase.
`std::counting_semaphore<N>` / `binary_semaphore`	`acquire()` decrements a count (blocking at 0), `release()` increments. Natural for "at most N concurrent users of a resource".
`std::stop_source` / `stop_token`	Cooperative cancellation signals (used by `jthread`).

11. Testing and Debugging Concurrent Code

11.1 ThreadSanitizer (your most important tool this week)

```
g++   -std=c++20 -g -O1 -fsanitize=thread -pthread prog.cpp -o prog && ./prog
clang++ -std=c++20 -g -O1 -fsanitize=thread -pthread prog.cpp -o prog && ./prog
# CMake (Week 5):  target_compile_options(t PRIVATE -fsanitize=thread)
#                  target_link_options(t PRIVATE -fsanitize=thread)
```

- Detects data races that actually *occur in that run's code paths* (even if the output was correct), plus lock-order inversions, use of destroyed mutexes, and more.
- Cost: roughly 5–15× slowdown and 5–10× memory. Use it in tests, not in production.
- It cannot detect races on code paths your tests never execute, and it can't find logical race conditions. Can't be combined with AddressSanitizer in the same build; run both in separate CI jobs.
- Alternatives: Valgrind's **Helgrind** and **DRD** (no recompilation needed, much slower).

11.2 Writing tests that can actually fail

1. **Stress it.** Run the operation millions of times from many threads; check invariants at the end (sums, counts, no duplicates, nothing lost).
2. **Start threads together** with a latch, barrier or `shared_future` "go" signal to maximize overlap.
3. **Run more threads than cores**, so preemption happens in the middle of your critical sections.
4. **Repeat the test** in a loop (`for i in $(seq 100); do ./test || break; done`) and under TSan.
5. **Test shutdown paths**: destroy objects while threads are blocked, close queues while full or empty, submit after shutdown.
6. **Put timeouts on tests** so a deadlock fails the test instead of hanging the CI forever.

11.3 Debugging

- **Hangs:** attach gdb/lldb, `thread apply all bt`, look for the wait-for cycle (§5.6).
- **Wrong results:** TSan first. Then add invariant assertions inside critical sections.
- **Heisenbugs:** adding `printf` changes timing and can hide a race. Prefer TSan and assertions; log into per-thread buffers if you must log.
- **Performance:** `perf stat`, `perf record`, and `perf c2c` (which detects false sharing) on Linux; Instruments on macOS. You'll use these more in Week 5.

Exercises §9–§11

1. Replace the mutex in a counter class with `std::atomic<long>`; benchmark mutex vs atomic seq_cst vs atomic relaxed with 1, 2, 4, 8 threads.
2. Implement `atomic_max` with CAS (without peeking at the listing), and test it under TSan.

3. Run `11_false_sharing.cpp` on a multi-core machine. Then try `alignas(32)` and `alignas(128)`. What changes?
4. Change `memory_order_release` to `memory_order_relaxed` in the writer of `10_atomics.cpp`. Is the program still correct? (Answer: no — explain why using Figure 5, even if it never fails on x86.)
5. Design the public interface (header only, no implementation) of a thread-safe LRU cache. Justify every method against the guidelines in §10.2.

12. Project: A Thread Pool

Per the course's working rule, this section gives you a specification, an interface, a design walkthrough, milestones, a test plan and a list of traps — but **not** the finished implementation. Everything you need has been covered in §2–§11. The skeleton below compiles; you write the bodies.

12.1 What a thread pool is and why it exists

Creating a thread per task is expensive (§2.6) and unbounded: 10,000 tasks would mean 10,000 threads fighting over a few cores. A **thread pool** creates a fixed set of **worker threads** once. Callers **submit** tasks into a shared **task queue**; idle workers take tasks out and run them; results flow back through **futures**. It is producer–consumer (§7) where the consumers are long-lived workers and the items are callables.

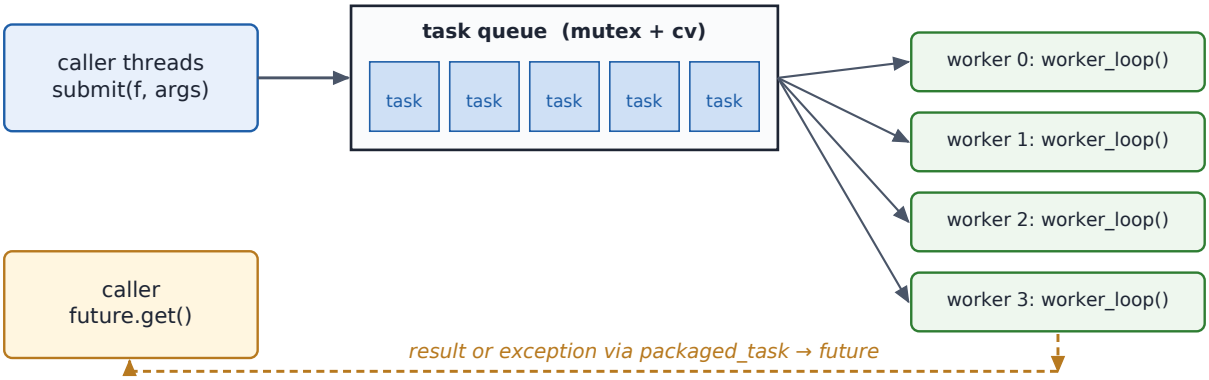


Figure 7. Thread pool architecture.

12.2 Requirements

#	Requirement
R1	Constructor starts N worker threads (N defaults to `hardware_concurrency()`, minimum 1).
R2	`submit(f, args...)` accepts any callable and arguments, returns `std::future<R>` where R is the call's result type (including `void`).
R3	Workers sleep (no busy-waiting) when the queue is empty and wake promptly when a task arrives.
R4	If a task throws, the exception reaches the caller via `future.get()`; the worker thread survives and keeps working.
R5	**Clean shutdown:** `shutdown()` and the destructor stop accepting new tasks, run every task already queued, and join all workers. No thread is ever left running or detached.
R6	`submit` after shutdown fails loudly (throws), never silently drops the task.
R7	`shutdown()` is idempotent: calling it twice, then destroying the pool, is safe.
R8	Zero TSan reports across the whole test suite; no deadlocks under stress.

12.3 The interface (compiles as-is)

thread_pool.hpp

```cpp
// include/my_project/thread_pool.hpp  --  SKELETON. You write the bodies.
#pragma once
#include <condition_variable>
#include <cstddef>
#include <functional>
#include <future>
#include <mutex>
#include <queue>
#include <thread>
#include <type_traits>
#include <vector>
class ThreadPool {
public:
    // Start `num_threads` workers (at least 1).
    explicit ThreadPool(std::size_t num_threads =
                            std::thread::hardware_concurrency());
    // Graceful: stop accepting work, finish every queued task, join all.
    ~ThreadPool();
    ThreadPool(const ThreadPool&) = delete;             // owns threads:
    ThreadPool& operator=(const ThreadPool&) = delete;  // not copyable
    ThreadPool(ThreadPool&&) = delete;                  // (moving a pool
    ThreadPool& operator=(ThreadPool&&) = delete;       //  is a stretch goal)
    // Queue f(args...) and return a future for its result. Exceptions
    // thrown by f are delivered through the future. Throws
    // std::runtime_error if the pool is shutting down.
    template <class F, class... Args>
    auto submit(F&& f, Args&&... args)
        -> std::future<std::invoke_result_t<F, Args...>>;
    // Stop accepting tasks, drain the queue, join workers. Idempotent.
    void shutdown();
    std::size_t size() const noexcept { return workers_.size(); }
private:
    void worker_loop();                         // what each thread runs
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;  // guarded by m_
    std::mutex m_;
    std::condition_variable cv_;                // "task available or stopping"
    bool stopping_ = false;                     // guarded by m_
};
```

12.4 Design walkthrough (read, then implement)

The worker loop

Each worker runs the same loop. In words — translating this into code is your job:

1. Lock the mutex with a `unique_lock`.
2. Wait on the CV until **"stopping, or there is a task"**. (Which of §6.3's rules does this predicate satisfy?)
3. If the queue is empty at this point, you were woken because of shutdown and there is nothing left to drain: return.
4. Otherwise move the front task out of the queue and pop it.
5. **Release the lock**, then run the task. Never run user code while holding the pool's lock (§4.4, §5.4).
6. Loop.

Notice that step 3 implements "drain before exit" for free: while tasks remain, workers keep taking them even after `stopping_` became true.

submit: from “any callable” to `std::function<void()>`

The queue stores one uniform type, `std::function<void()>`, but callers submit callables with arbitrary arguments and return types and want a typed future back. The bridge:

1. Compute the result type: `using R = std::invoke_result_t<F, Args...>;`
2. Bind the callable and its arguments into a nullary callable (a lambda capturing them, or `std::bind`).
3. Wrap that in a `std::packaged_task<R()>` and take its `get_future()` **before** queuing it.
4. Queue a `void()` callable that invokes the packaged task. Because `packaged_task` already captures exceptions into the future, R4 is satisfied automatically.
5. Under the lock: if `stopping_`, throw (R6); else push. After unlocking: `notify_one()`.

> **PITFALL**
>
> `std::function` **requires a copyable callable, and** `std::packaged_task` **is move-only.** Pushing a lambda that captures a `packaged_task` by move won’t compile. Two standard fixes: hold the task in a `std::shared_ptr<std::packaged_task<R()>>` and capture the shared_ptr (copyable), or, in C++23, make the queue store `std::move_only_function<void()>`. Pick one and be able to explain why it is needed.

shutdown

Set `stopping_ = true` **under the lock** (otherwise: lost wakeup — a worker could check the predicate, miss the flag, and sleep forever), then `notify_all()` (every worker must wake to see it), then `join()` each worker that is still joinable. Make a second call harmless (R7). The destructor just calls `shutdown()`.

12.5 Milestones

#	Milestone	Done when
M1	Fixed workers + queue of `std::function<void()>`; `submit` returns `void`; destructor drains and joins.	10,000 tasks incrementing an atomic counter → counter == 10,000 after the pool is destroyed.
M2	Typed `submit` returning `std::future<R>` via `packaged_task`.	Submitting `[](int x){ return x*x; }` with 1..1000 and summing `get()`s gives the right total. `void` tasks work.
M3	Exceptions and shutdown semantics (R4–R7).	A throwing task’s future rethrows; later tasks still run; submit-after-shutdown throws; double shutdown is safe.
M4	Organize as a library: `include/`, `src/`, `tests/`, `app/` per the course structure (the template member goes in the header).	Builds as a separate library target (CMake comes properly in Week 5; a Makefile is fine now).
M5	Test suite under ThreadSanitizer (12.6).	All tests pass 100× in a row with `-fsanitize=thread`.
M6	Benchmark (12.7).	A table of time vs thread count, and a written explanation of the curve.

12.6 Test plan

1. **Basic correctness:** many small tasks, check all results.
2. **Void tasks and move-only arguments/results** (e.g. a task returning `std::unique_ptr<int>`).
3. **Exception propagation:** a task throws `std::runtime_error`; `get()` rethrows it; the pool is still usable.
4. **Graceful drain:** submit 1,000 tasks that each sleep 1 ms into a 2-thread pool, destroy the pool immediately, verify all 1,000 ran.
5. **Submit after shutdown** throws.
6. **Many concurrent submitters:** 8 threads each submitting 10,000 tasks at once (start them with a latch).
7. **Idle pool:** construct and destroy a pool with no tasks; must not hang.
8. **Single-thread pool:** tasks run one at a time in FIFO order (verify order).
9. **Timeouts:** wrap each test so a deadlock fails instead of hanging (e.g. `future.wait_for(5s)`).

12.7 Benchmark and analysis

Measure a CPU-bound workload (e.g. 10,000 tasks each doing a few hundred microseconds of arithmetic) with 1, 2, 4, 8, 16 workers, and compare to (a) a plain serial loop and (b) one `std::async` per task. Also try very tiny tasks (a single addition): the pool will be *slower* than serial. Explain why: queue locking, CV wake-ups and cache traffic cost more than the work. The program below shows the plain-threads version of the measurement for reference:

14_scaling.cpp

```cpp
// 14_scaling.cpp  --  split work, measure speedup
#include <chrono>
#include <cmath>
#include <iostream>
#include <thread>
#include <vector>
double work(long lo, long hi) {            // CPU-bound dummy work
    double s = 0;
    for (long i = lo; i < hi; ++i) s += std::sqrt(double(i));
    return s;
}
int main() {
    const long N = 200'000'000;
    for (unsigned T : {1u, 2u, 4u, 8u, 16u}) {
        auto t0 = std::chrono::steady_clock::now();
        std::vector<double> partial(T);    // each thread writes its own slot
        std::vector<std::thread> ts;
        for (unsigned t = 0; t < T; ++t)
            ts.emplace_back([&, t] {
                long lo = N * t / T, hi = N * (t + 1) / T;
                partial[t] = work(lo, hi);
            });
        for (auto& th : ts) th.join();
        double total = 0;
        for (double p : partial) total += p;
        std::chrono::duration<double> d =
            std::chrono::steady_clock::now() - t0;
        std::cout << T << " threads: " << d.count() << " s  (sum "
                  << total << ")\n";
    }
}
```

In your write-up, connect the curve to Amdahl's law (§1.4): where is the serial fraction? (Hint: the single queue mutex.) Why does going beyond the number of cores not help? This is exactly the Week 6 question, asked early.

12.8 Traps (check your code against each)

- Running the task **while holding** the queue lock — the pool becomes serial and any task that submits a new task deadlocks.
- Setting `stopping_` **without** the lock, or notifying with `notify_one` on shutdown — some workers sleep forever.
- Worker loop exits when `stopping_` is true **even though tasks remain** — breaks graceful drain (R5).
- Waiting without a predicate — spurious wakeups pop from an empty queue (UB).
- Capturing locals **by reference** in a submitted lambda and returning before the task runs — dangling reference.
- **Pool-induced deadlock:** a task that blocks on the future of another task submitted to the *same* pool. If every worker is busy waiting, nobody runs the inner tasks. With a 1-thread pool it deadlocks every time. Document this restriction (or, as a stretch, let waiting workers run queued tasks).
- Destroying the pool **from inside one of its own tasks**: the destructor would `join()` the calling thread itself (`std::system_error` with `resource_deadlock_would_occur`).
- Letting an exception escape `worker_loop` — `std::terminate`. With `packaged_task` this can't happen; with raw `std::function` tasks it can.

12.9 Stretch goals

- `wait_idle()`: block until the queue is empty and no task is running (needs an "active tasks" counter and a second CV).
- Bounded task queue with back-pressure, reusing your `BoundedQueue` from §7.
- Use `std::jthread` + `std::stop_token`, and offer `shutdown_now()` that discards pending tasks (their futures get `broken_promise`).
- Per-worker queues with **work stealing** (how production pools such as TBB and Go's scheduler reduce contention on one queue).
- Task priorities with `std::priority_queue`.
- Metrics: tasks completed per worker, average queue wait time.

12.10 Self-review rubric

Area	Excellent	Needs work
Correctness	All requirements met; TSan clean over 100 runs.	Occasional hangs or TSan reports.
Lifetime / RAII	No manual unlocks; threads always joined; idempotent shutdown.	Manual lock/unlock; detach; double-join crash.
Lock discipline	Tiny critical sections; tasks run unlocked; state changes under lock.	Task runs under lock; flags set outside lock.
API design	Typed futures; clear documented guarantees and restrictions.	Void-only API; silent task drop after shutdown.
Testing	Stress, shutdown, exception and timeout tests.	Only a happy-path demo in `main`.
Analysis	Benchmark table + explanation using Amdahl and contention.	Numbers with no explanation.

13. Review: Cheat Sheet and Self-Check

13.1 “I need to …” → use

I need to…	Use
Run a function on another thread	`std::jthread` (C++20) or `std::thread` + join / RAII guard
Get a result or exception back	`std::async(std::launch::async, ...)`, `packaged_task`, or `promise`
Protect shared data / an invariant	`std::mutex` + `std::lock_guard` / `std::scoped_lock`
Lock two or more mutexes	`std::scoped_lock lk(a, b);`
Wait until a condition is true	`std::condition_variable` + `unique_lock` + `wait(lk, pred)`
Many readers, rare writers	`std::shared_mutex` + `shared_lock` / `unique_lock` (measure first)
A counter or flag	`std::atomic<T>` (default ordering; `relaxed` for pure counters)
Publish data with a flag, lock-free	release store / acquire load
Initialize something once	function-local `static`, or `std::call_once`
Per-thread state	`thread_local`
Pass work between threads	bounded blocking queue with `close()`
Wait for N threads to finish a step	`std::latch` (once) / `std::barrier` (repeating)
Limit concurrent access to N	`std::counting_semaphore<N>`
Stop a thread	`jthread::request_stop()` + polling `stop_token` (cooperative)
Find races	`-fsanitize=thread`

13.2 Golden rules

1. Prefer not sharing over sharing, and sharing immutable data over sharing mutable data.
2. Every piece of shared mutable state has exactly one mutex guarding it; write down which.
3. Never `lock()`/`unlock()` by hand; never let a reference to guarded data escape the lock.
4. Never call unknown code, do I/O, or block on another thread while holding a lock.
5. Lock multiple mutexes together or in one global order.
6. Condition variables: change state under the lock; always wait with a predicate.
7. Every blocking operation needs a shutdown path.
8. Default to `seq_cst` atomics; anything weaker needs a happens-before argument.
9. Atomic variables don’t make multi-step algorithms atomic.
10. TSan-clean, stress-tested, or it isn’t done.

13.3 Self-check questions

Try to answer in one or two sentences each, then check §13.4.

Q1. What happens if a `std::thread` object is destroyed while joinable?

Q2. Why does `std::thread t(f, x)` not let `f` modify `x` when `f` takes `int&`?

Q3. Define a data race. What is its consequence?

Q4. Give an example of a race condition that is not a data race.

Q5. When must you use `unique_lock` instead of `lock_guard`?

Q6. Name the four deadlock conditions, and the one that lock ordering breaks.

Q7. Why must the CV predicate be modified under the mutex?

Q8. Why always wait with a predicate?

Q9. What does `std::async(std::launch::async, f);` (result discarded) do?

Q10. What does a release store paired with an acquire load guarantee?

Q11. What is false sharing and how do you fix it?

Q12. Why is `if (!s.empty()) { x = s.top(); s.pop(); }` unsafe even if each call locks?

Q13. In a thread pool, why run the task after releasing the lock?

Q14. Why does shutdown use `notify_all` rather than `notify_one`?

13.4 Answers

A1. The program calls `std::terminate()`.

A2. The constructor copies `x` into the thread's own storage; it won't even compile. Use `std::ref(x)`.

A3. Concurrent accesses to the same location, at least one a write, at least one non-atomic, without happens-before ordering. Undefined behaviour.

A4. Check-then-act with individually locked calls, e.g. `if (acct.balance() >= x) acct.withdraw(x);`.

A5. With condition variables, and whenever you need to unlock early, defer, try, time out, or move the lock.

A6. Mutual exclusion, hold-and-wait, no preemption, circular wait. Ordering breaks circular wait.

A7. Otherwise the change and notify can fall between the waiter's check and its sleep: a lost wakeup.

A8. Spurious wakeups and stolen wakeups: `wait` can return when the condition is false.

A9. Runs synchronously: the temporary future's destructor blocks until `f` finishes.

A10. If the load sees the stored value, everything before the store happens-before everything after the load.

A11. Independent variables written by different cores share a cache line, which ping-pongs. Pad/align to separate lines.

A12. Another thread can change the stack between calls; fuse into `try_pop()`.

A13. Otherwise only one task runs at a time and tasks that touch the pool deadlock.

A14. Every sleeping worker must wake to observe `stopping_` and exit.

14. Curated Resources

These are the best sources for this week's material, chosen for accuracy and depth. You do not need all of them. The **Core** list is enough for everything in this guide; use the rest to go deeper on specific topics.

14.1 Core (use these this week)

Resource	Why / what to read
Anthony Williams, *C++ Concurrency in Action*, 2nd ed. (Manning, 2019)	The definitive book; Williams maintained Boost.Thread and helped write the C++ threading library. Read ch. 1–4 for this week, ch. 5 for the memory model, ch. 9.1 for thread pools, ch. 11 for testing and debugging.
cppreference — Concurrency support library en.cppreference.com/w/cpp/thread	The precise reference for every class here, with examples. Check every function you use; note the "since C++XX" tags.
OSTEP (Arpaci-Dusseau), free online pages.cs.wisc.edu/~remzi/OSTEP/	The Concurrency part (ch. 26–32): threads, locks, lock-based data structures, condition variables, semaphores, common bugs. In C with pthreads, but the concepts map 1:1, and it previews your OS course.
Arthur O'Dwyer, "Back to Basics: Concurrency", CppCon 2020 (YouTube)	One hour; the clearest tour of mutex, CV, `once_flag`, `promise`/`future` and C++20 additions.
David Olsen, "Back to Basics: C++ Concurrency", CppCon 2023 (YouTube)	A more recent beginner-friendly overview, including `jthread` and C++20 primitives.

14.2 Atomics and the memory model

Resource	Why / what to read
Jeff Preshing's blog preshing.com	"An Introduction to Lock-Free Programming", "Acquire and Release Semantics", "Memory Ordering at Compile Time", "The Synchronizes-With Relation". Short, illustrated, excellent.
Herb Sutter, "atomic<> Weapons" (C++ and Beyond 2012), parts 1–2 (YouTube)	Three hours on the memory model, still the best explanation of *why* it is designed the way it is.
Fedor Pikus, "C++ atomics, from basic to advanced", CppCon 2017 (YouTube)	What atomics actually cost on hardware, with benchmarks.
Mara Bos, *Rust Atomics and Locks* (O'Reilly, 2023), free at marabos.nl/atomics	Written for Rust, but Rust uses the C++ memory model; ch. 2–3 are among the clearest explanations of orderings, and later chapters build a mutex, CV and channel from scratch.

14.3 Design, practice and going further

Resource	Why / what to read
C++ Core Guidelines, section CP (Concurrency and parallelism) isocpp.github.io/CppCoreGuidelines	Short, actionable rules (e.g. CP.20 “Use RAII, never plain lock()/unlock()”, CP.42 “Don’t wait without a condition”).
Bryce Adelstein Lelbach, “The C++20 Synchronization Library”, CppCon 2019 (YouTube)	`latch`, `barrier`, `semaphore`, `atomic::wait`, and building a task system with them.
Anthony Williams, “More Concurrent Thinking in C++: Beyond the Basics”, CppCon 2022 (YouTube)	Practical patterns once the basics are solid.
Rainer Grimm’s blog modernescpp.com	Many short posts on every concurrency feature, with runnable examples.
Clang ThreadSanitizer docs clang.llvm.org/docs/ThreadSanitizer.html	Flags, suppressions, and limits of TSan.
Paul McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?* (free PDF)	Deep, hardware-aware; by the Linux RCU maintainer. For later, alongside Computer Architecture.
Herlihy & Shavit, *The Art of Multiprocessor Programming* (2nd ed., 2020)	The theory of concurrent data structures and correctness (linearizability). For Distributed Systems later.

RULE OF THUMB

Use resources to understand; write the code yourself. A good loop for each topic: read the section here, watch or read one external source on it, close everything, implement the example from memory, run it under TSan.

14.4 Looking ahead

Week 5 puts your thread pool into a proper CMake project with tests, Debug/Release and sanitizer builds, and teaches profiling. **Week 6** combines the bounded queue, the thread pool and futures into the concurrent data processing engine and asks you to explain its scaling curve — which you now have the vocabulary for: serial fraction, lock contention, false sharing, memory bandwidth and oversubscription.